AI Report — DD2 Project 1, Option 2 **CSCE 3304 Digital Design II, Fall 2026 · Group 5** Salma Elmarakby (900232658) · Mikhael Khalil (900232633) · Marina Mikhael (900233146) AI used as a development aid (overview, section 4.1): no model runs when the tool runs.

1. How we used AI

We wrote `AGENTS.md` before any code (commit `2cd4d9b`). It tells the assistant how the project is checked (`make check`, `make test`, `make all`), the rules of overview section 4.3, and what it must never do: weaken a check, edit the check scripts, broken cases or the PDK, fabricate numbers, move tags, or commit large files. Every AI-assisted change is logged in `ai/use_log.md` (CP1), `ai/use_log_cp2.md` (CP2) and `ai/use_log_final.md` (final), 23 entries in all, and every mistake our checks caught is in the three matching mistakes logs, 22 entries in all.

Part	What the AI did	What we did
Repository, Makefile, tool wrapper (CP1)	Drafted `AGENTS.md`, the Makefile targets and `src/runner.py` (L-001, L-002)	Fixed the rules and layout to the handout; removed hard-coded WSL paths; checked the links `make setup` makes
AIG extraction, cuts, classes (CP1)	Wrote `extract_aigs.py`, the AIGER parser, the cut enumerator and the classifier (L-003 to L-005)	Found the `dch` choice-node error (M-01) and the dropped trivial cut (M-02) by comparing counts; specified the simulation check every cut must pass
Table 1, HD classes, Table 4 (CP1)	Wrote the Table 1 runner, the HD class table and the redundancy checker (L-006 to L-008)	Audited the transistor counts by hand (M-03) and rejected a checker that matched Table 4 with hard-coded truth tables (M-04)
Costing, area model, ranking, Liberty, Table 5 (CP2)	Wrote `cmos_cost.py`, `area_model.py`, `candidates.py`, `liberty_writer.py`, `table5.py`, `equivalence.py`, `netlist_repair.py` (L-101 to L-107)	Chose the construction space and the model form; insisted that every mapping is checked for equivalence and that every check has a positive and a negative control (M-103 to M-105)
CP2 audit	Audited the repository from a clean clone (L-108)	Decided what the README may claim; found that the results depend on the Yosys build (M-109)
Final flow	Wrote steps 10 to 12, the curve, OpenSTA delay, gate-level testbenches, real-cell intake, the full demo, the tests and broken cases, and drafts of both reports (L-201 to L-206)	Decided the interpretation of the greedy rule, required that a final remap reproduce the greedy total, read every result table, and corrected the report where the data did not support a claim

2. Tools, models and cost

Phase	Tool	Model
CP1	Antigravity	Gemini 3.8 Flash, Gemini 4 Argon
CP2 and final	Claude Code (CLI, VS Code extension)	Claude Opus 5.5 (`claude-opus-5-5`)

Cost: **[to be completed by the group: subscription or API spend per tool]**. No AI runs inside the tool, so the flow itself costs nothing to run and gives the same output for the same input; we confirmed that by repeated mappings (CP2, L-108) and by requiring the final greedy total to be reproduced exactly by a separate mapping run.

3. How we described tasks and checked the output

We described each task by pointing at the handout section it implements and at the check it must pass, never by asking for a number. Examples: "Reproduce Table 4 with the tests of Table 3; the counts must come out of the code, not out of a list"; "Every mapped netlist must be proved equivalent to the design before ABC; show that the check fails on a netlist with one gate swapped". The output was accepted only through scripts:

- every cut is re-simulated on all 2^k inputs (335,126 cuts);
- the redundancy checker must classify all 264 combinational HD cells as Exists and reproduce all 9 counts of Table 4;
- every mapped netlist must pass ABC `dsec` (or `cec` for combinational designs) against the design before ABC, and SPM and the multiplier must pass the tutorials' testbenches;
- every Liberty file must load in Yosys with exactly 189 + N cells, and every new cell's function must evaluate to its class;
- the cell-response reader must reject failed, wrong and malformed cells;
- each of these checks has a broken case in `tests/broken/` that must fail (section 4).

4. How we applied the rules of section 4.3

1. **Pass or fail from check scripts.** Every pass/fail in the flow is a return code or a string from Yosys, ABC, Icarus or our own test scripts; `make test` and `make all` stop on the first failure.
2. **Every number from a file the flow wrote.** Every number in both reports is copied from a file in `results/`, regenerated in one clean run on Yosys 0.70+201. When a README number disagreed with the results files, the README was corrected (M-108), and the CP2 numbers on another Yosys build were saved separately (`results/rerun_yosys_0.70_201/`) instead of being mixed in.
3. **A broken case per check.** Ten checks, ten broken cases (technical report, section 3), including two added in the final phase when a broken case turned out to be too weak (M-202) or a check path was new (M-207).
4. **AI does not change checks, broken cases or the PDK.** `AGENTS.md` forbids it. When a check failed during the final phase (the multiplier failing `dsec`), the fix added a combinational path (`cec`) with its own new broken case; the existing check and its broken case were left unchanged. The PDK is reached only through the `pdk` link and read.
5. **Every member can explain every part. [to be completed by the group: how the code was walked through among the members before the interview]**

5. The five most instructive AI mistakes

1. Matching a reference by hard-coding it (M-04, CP1). The first Table 4 implementation compared tuples lexicographically, ignored the ≤ 3-in-series rule, and then used a hard-coded list and one hard-coded truth table (`0x017F`, a 14-transistor gate) so that its counts equalled the reference 5 and 8. All tests passed. We caught it by reviewing the nine single-stage New classes by hand against the rule of section 4 step 4. *A weaker check would have missed it:* a test that compares counts with the reference cannot tell a correct count from a forced one. This is the mistake that shaped how we wrote every later prompt: the number must come out of the method.

2. A restructuring that made the AIG larger (M-01, CP1). `strash; dch; write_aiger` wrote ABC's choice nodes into the file, so cpu8 went from 932 to 1,702 AND nodes "after optimization". Caught by reading `results/aig_stats.md`; a second `strash` gives 849. *Weaker check:* yes; both tools exited 0 and wrote valid AIGER files.

3. An equivalence check that changed what it checked (M-105, CP2). Writing the reference netlist inside the mapping run renamed signals and changed ABC's result (cpu8 script A 4,160.2 instead of 4,221.5 um2 on that build), so the measured design was no longer the one of section 2. Caught by comparing with `given/scripts/map.sh`. *Weaker check:* yes; the mapping was still correct and passed `dsec`.

4. Results that depend on the Yosys build (M-109, CP2). The CP2 Table 5 was measured on Yosys 0.70+190 and gave +0.34%; the same tag on 0.70+201 gives +0.05% and a different tenth cell. Caught by rerunning a clean clone on a second machine. *Weaker check:* yes; `make test` and `make all` pass on both builds, because they check equivalence, not areas. It is why the final results come from one clean run on one build, and why the report treats savings below about 0.1% as noise.

5. A delay table full of plausible-looking wrong numbers (M-206, final). The OpenSTA parser read the first path of the report, which for designs with asynchronous resets is a reset recovery check: uart_apb's worst path was reported as 0.0 ns. Caught because a 4,673-AND design cannot have a zero-delay critical path; the log showed the path group. *Weaker check:* yes; OpenSTA exited 0 and every row had a number.

A sixth deserves mention because it is about the checks themselves: the first broken case for the gate-level testbench only asserted that the run did not pass, which a missing file would also satisfy (M-202). Broken cases now assert the specific failure (`ERROR:` from the testbench, `NOT EQUIVALENT` from ABC).

6. One decision we would never hand to an AI

Whether a cell is worth adding. The AI can rank, map and measure, but the conclusion "these cells save area" rests on judging whether a 0.08% change is a saving or noise, and an AI asked to "find cells that save area" is rewarded for reporting one. Our flow kept 9 cells whose total gain was 66 um2, and four of them are never instantiated in any final mapping: their "gain" is ABC reacting to a changed library. The check that catches an AI getting this wrong is in the flow: `results/greedy.md` lists every kept cell with zero instances in the final mappings, Table 5 reports instances next to every

area, and the 30% experiment shows how much of the result is the area estimate (one cell 30% smaller turns 0.08% into 0.70%).

7. Where AI saved time and where it cost time

Saved: boilerplate and plumbing (the WSL tool bridge, Liberty writing, CSV/markdown reports), and fast exploration of ABC and OpenSTA behaviour (the `dsec` flip-flop pairing, the Liberty `ff` models, the path groups). The final phase (greedy selection, real-cell intake, 30% experiment, curve, delay, gate-level testbenches, demo, tests) was written and run in one session.

Cost: every mistake above cost a rerun, and the costliest were silent ones: the hard-coded Table 4 (M-04) and the build dependence (M-109) each required re-deriving results we had believed. Two final-phase mistakes (M-203, M-205) came from scripted multi-line edits that silently did nothing or broke a string literal; we stopped using them. **[to be completed by the group: hours estimate per phase]**

8. What we would and would not hand to an AI next time

Would: tool wrappers, report writers, test scaffolding, and first implementations of well-defined algorithms (cut enumeration, canonical classes), provided the check is written first and the AI is told the check. **Would not:** the choice of what to measure and what a result means (section 6), any code whose output is compared with a reference count without an independent derivation (M-04), and the interpretation of tool output whose format we have not read ourselves (M-206). Next time we would also pin the tool versions in `make check` from the first day, so that a result can be tied to the build that produced it.